

IsoQuant: Hardware-Aligned $SO(4)$ Isoclinic Rotations for LLM KV Cache Compression

Zhongping Ji

Abstract

Orthogonal feature decorrelation is a powerful primitive for low-bit online vector quantization, but the standard recipe of applying a dense random orthogonal matrix incurs prohibitive $O(d^2)$ compute and storage cost. Recent work such as RotorQuant replaces the global rotation with blockwise 3D Clifford rotors, substantially reducing complexity. However, the resulting 3D partition is awkward for modern hardware because neural features are usually powers of two in width, leading to tail handling, misaligned memory access, and limited local mixing capacity.

We propose **IsoQuant**, a new 4D block rotation framework based on quaternion algebra and the isoclinic decomposition of $SO(4)$. Each block of four coordinates is identified with a quaternion and rotated by a closed-form sandwich product $T(v) = q_L v \bar{q}_R$ parameterized by a pair of unit quaternions. This yields two practical variants: *IsoQuant-Full*, which uses the full six degrees of freedom of $SO(4)$, and *IsoQuant-Fast*, which restricts to a single isoclinic factor for lower overhead. The same framework also admits a lightweight 2D planar special case, which we treat as a degenerate operating point rather than the main method. At $d = 128$, IsoQuant-Full reduces forward rotation cost from roughly 2,408 FMAs in RotorQuant to 1,024, while IsoQuant-Fast further reduces it to 512. Across 18 fused CUDA benchmark settings covering $d \in \{128, 256, 512\}$, bit width $\{2, 3, 4\}$, and both FP16/FP32 execution, IsoQuant-Full, IsoQuant-Fast, and the 2D special case achieve mean kernel-level speedups of $4.49\times$, $4.66\times$, and $4.66\times$ over RotorQuant while maintaining comparable reconstruction MSE, with peak speedups above $6\times$. At present, however, our empirical validation is limited to the stage-1 quantize-dequantize path on synthetic normalized vectors; we have not yet validated end-to-end KV-cache quality on real model activations. Code: <https://github.com/ParaMind2025/isoquant>.

1 Introduction

KV-cache compression has emerged as a key systems bottleneck for long-context large language model inference. A core insight behind online vector quantization methods such as TurboQuant [1] is that feature decorrelation before scalar quantization dramatically improves rate-distortion behavior. In particular, a random orthogonal transform spreads information across coordinates so that per-coordinate Lloyd-Max quantization becomes far more effective than quantizing the original basis directly.

The main drawback is cost. For a head dimension d , a dense orthogonal transform requires $O(d^2)$ parameters and arithmetic, which is difficult to justify in latency-sensitive settings such as autoregressive decoding. RotorQuant [2] addresses this issue by replacing the dense transform with sparse blockwise rotations in the geometric algebra $Cl(3, 0)$. This preserves the qualitative idea of local decorrelation while reducing complexity to linear in d .

Despite this progress, the 3D block structure leaves performance on the table. First, it is not hardware aligned: common head dimensions such as 64, 128, and 256 are powers of two, so partitioning into triples creates awkward remainder handling and non-ideal memory layout. For example, $d = 128$ yields forty-two full 3D blocks plus one 2D tail. Second, a 3D rotation has only three intrinsic degrees of freedom, which limits how aggressively each local subspace can mix correlated coordinates.

This paper explores a different operating point. We move from 3D Clifford blocks to 4D quaternion blocks and leverage the Lie-theoretic decomposition

$$so(4) \cong su(2) \oplus su(2), \quad (1)$$

which implies that every 4D rotation can be represented by a pair of unit quaternions acting from the left and right. This gives a closed-form, low-overhead parameterization of $SO(4)$ that is both mathematically clean and implementation friendly. We therefore name the method **IsoQuant**, emphasizing the isoclinic structure that underlies its blockwise $SO(4)$ transform.

Contributions. We make four contributions.

1. We introduce **IsoQuant**, a 4D block rotation scheme for online vector quantization based on quaternion sandwich products.
2. We derive two practical 4D instantiations, **IsoQuant-Full** and **IsoQuant-Fast**, and show that the same framework also admits a **lightweight 2D planar special case** as an additional operating point.
3. We analyze why 4D blocks are especially attractive for systems deployment: they improve arithmetic efficiency, eliminate most boundary handling, and align naturally with SIMD and fused-kernel execution.
4. We provide a fused CUDA implementation and show in fair kernel-level comparisons against RotorQuant fused CUDA kernels that IsoQuant achieves consistent speedups while preserving reconstruction quality.

2 Related Work

Online vector quantization and KV-cache compression. TurboQuant [1] frames online vector quantization as a dense-rotation-plus-scalar-quantization pipeline and combines it with QJL residual correction [3]. RotorQuant [2] reduces the heavy dense transform by replacing it with sparse 3D Clifford rotors. Other KV-cache compression methods such as KIVI [4] and KVQuant [5] focus on calibration, asymmetric quantization, or hardware-aware packing rather than explicit geometric decorrelation.

Quaternion and geometric representations. Unit quaternions are a classical tool for representing rotations [6]. In machine learning, algebraically structured representations have appeared in equivariant models, geometric neural networks, and efficient transformations on structured features. RiemannFormer [7] provides a recent geometric treatment that explicitly discusses the isoclinic decomposition of $SO(4)$ in the setting of curved-space attention. Our use of this structure is narrower and more systems-oriented: we exploit the same $SO(4)$ factorization to design a cheap blockwise decorrelating transform for low-bit quantization.

3 Problem Setup

We consider the stage-1 mean-squared-error component of online vector quantization. Given an input vector $x \in \mathbb{R}^d$, we seek an encoder E , scalar quantizer Q , and decoder D such that

$$\hat{x} = D(Q(E(x))) \quad (2)$$

minimizes reconstruction error while keeping both parameter count and online compute small.

As in prior work [1, 2], we factor the problem into three pieces:

1. an orthogonal or approximately orthogonal transform that decorrelates coordinates;
2. coordinate-wise scalar quantization in the transformed basis;
3. an inverse transform to reconstruct the vector.

To stabilize the scalar quantizer, we additionally separate norm and direction. Writing

$$x = \rho \bar{x}, \quad \rho = \|x\|_2, \quad \|\bar{x}\|_2 = 1, \quad (3)$$

we quantize the normalized direction $\bar{x}$ and store or transmit the norm ρ separately. This follows the implementation pattern used by efficient quantizers and keeps the transformed coordinates within a predictable dynamic range.

4 Quaternion and $SO(4)$ Preliminaries

We begin by identifying each 4D block with a quaternion

$$v = x_0 + x_1 \mathbf{i} + x_2 \mathbf{j} + x_3 \mathbf{k} \in \mathbb{H}, \quad (4)$$

where $\mathbb{H}$ denotes the quaternion algebra and $\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1$ [6]. For

$$q = a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}, \quad (5)$$

its quaternion conjugate is

$$\bar{q} = a - b\mathbf{i} - c\mathbf{j} - d\mathbf{k}. \quad (6)$$

Unit quaternions form the 3-sphere S^3 and provide a nonsingular parameterization of rotational factors that will be central to our construction.

The relevant geometric structure is the special orthogonal group in four dimensions. At the Lie-algebra level one has the classical decomposition

$$\mathfrak{so}(4) \cong \mathfrak{su}(2)_L \oplus \mathfrak{su}(2)_R, \quad (7)$$

which underlies the isoclinic decomposition of $SO(4)$ discussed, for example, in RiemannFormer [7]. This splitting implies that every infinitesimal 4D rotation can be written as the sum of two commuting components. More precisely, if

$$X = X_A + X_B, \quad X_A \in \mathfrak{su}(2)_L, \quad X_B \in \mathfrak{su}(2)_R, \quad (8)$$

then $[X_A, X_B] = 0$, and therefore the corresponding group element factors as

$$R = \exp(X) = \exp(X_A + X_B) = \exp(X_A) \exp(X_B), \quad (9)$$

where the exponential map is given by

$$\exp : \mathfrak{so}(4) \rightarrow SO(4), \quad X \mapsto \exp(X) = \sum_{k=0}^{\infty} \frac{X^k}{k!}. \quad (10)$$

This commuting factorization is the Lie-theoretic counterpart of the left/right isoclinic decomposition.

Proposition 1. *Let $q_L, q_R \in S^3$ be unit quaternions. Then the map*

$$T(v) = q_L v \overline{q_R} \quad (11)$$

defines an orthogonal transformation of $\mathbb{R}^4$. Its inverse is

$$T^{-1}(v) = \overline{q_L} v q_R, \quad (12)$$

and the pairs (q_L, q_R) and $(-q_L, -q_R)$ induce the same element of $SO(4)$.

Proof sketch. Quaternion multiplication by a unit quaternion preserves the Euclidean norm on $\mathbb{H} \cong \mathbb{R}^4$, both for left multiplication and for right multiplication by the conjugate. Hence the composition $v \mapsto q_L v \overline{q_R}$ is norm-preserving and therefore orthogonal. The inverse follows immediately from associativity of quaternion multiplication together with $q_L \overline{q_L} = \overline{q_R} q_R = 1$. Finally, replacing both q_L and q_R by their negatives leaves the map unchanged because

$$(-q_L) v \overline{(-q_R)} = (-q_L) v (-\overline{q_R}) = q_L v \overline{q_R}. \quad (13)$$

□

The left and right quaternion factors correspond to the left-isoclinic and right-isoclinic components, respectively. Hence a pair of unit quaternions provides a compact, closed-form parameterization of a general 4D rotation, up to the standard double-cover ambiguity. This parameterization is especially attractive for quantization because it realizes the full six degrees of freedom of $SO(4)$ without ever requiring explicit storage of a dense 4×4 orthogonal matrix.

4.1 Block-Diagonal Rotations in $SO(4g)$

Let

$$g = \left\lceil \frac{d}{4} \right\rceil. \quad (14)$$

After zero padding when necessary, the input is naturally viewed as an element of $\mathbb{R}^{4g}$. In a fully dense formulation one would apply an unrestricted orthogonal transform in $SO(4g)$, which plays the role of a padded surrogate for the original $d \times d$ orthogonal rotation. IsoQuant instead restricts this transform to a structured Lie subgroup obtained from independent 4D rotational blocks.

Formally, define the Lie subalgebra

$$\mathfrak{g} = \bigoplus_{i=1}^g \mathfrak{so}(4)^{(i)} \subset \mathfrak{so}(4g). \quad (15)$$

Any element $H \in \mathfrak{g}$ has block-diagonal form

$$H = \text{diag}(X_1, X_2, \dots, X_g), \quad (16)$$

where each $X_i \in \mathfrak{so}(4)$ and all off-diagonal blocks vanish. Since the exponential map preserves block-diagonal structure, one obtains

$$\exp(H) = \text{diag}(\exp(X_1), \exp(X_2), \dots, \exp(X_g)). \quad (17)$$

Because $\exp(X_i) \in SO(4)$ for each block, the resulting matrix $\exp(H)$ belongs to $SO(4g)$. The set of all such matrices forms an embedded Lie subgroup isomorphic to

$$SO(4) \times SO(4) \times \dots \times SO(4), \quad (18)$$

with g factors. In this sense, IsoQuant replaces a dense high-dimensional orthogonal transform by a block-diagonal subgroup of $SO(4g)$ composed of local $SO(4)$ actions.

The matrix-exponential viewpoint is primarily conceptual: it makes clear which subgroup of $SO(4g)$ is being used and how this subgroup arises from a direct-sum Lie algebra. In the actual computation, however, we do not construct the skew-symmetric blocks X_i or evaluate the corresponding matrix exponentials. Each local $SO(4)$ action is instead realized more efficiently by a quaternion pair $(q_L^{(i)}, q_R^{(i)})$, which yields the same block rotation in closed form while avoiding dense matrix materialization.

5 Method: IsoQuant

5.1 Blockwise 4D Rotation

We now instantiate the subgroup construction above as a stage-1 quantization transform. Let $\bar{x} \in \mathbb{R}^d$ denote the normalized input direction. We partition $\bar{x}$ into

$$g = \left\lceil \frac{d}{4} \right\rceil \quad (19)$$

blocks,

$$\bar{x} = \left[v^{(1)}, v^{(2)}, \dots, v^{(g)} \right], \quad (20)$$

where each $v^{(i)} \in \mathbb{R}^4$ is identified with an element of $\mathbb{H}$. If d is not divisible by 4, the final block is zero padded, so the full transformed object lies in $\mathbb{R}^{4g}$.

From the perspective of Section 4.1, the encoder acts on the padded vector by an element of the block-diagonal subgroup $(SO(4))^g \subset SO(4g)$. The algorithm does not materialize this action as a dense matrix. Instead, each local $SO(4)$ factor is represented by one or two unit quaternions and applied directly in closed form.

Accordingly, each block undergoes the sequence

$$v^{(i)} \mapsto \tilde{v}^{(i)} \mapsto \hat{v}^{(i)} \mapsto v_{\text{rec}}^{(i)}. \quad (21)$$

That is, one first applies a local $SO(4)$ rotation, then performs coordinate-wise scalar quantization in the rotated basis, and finally applies the inverse local rotation. The recovered blocks are concatenated and the original norm ρ is restored.

5.2 IsoQuant-Full

IsoQuant-Full uses the complete double-sided action of $SO(4)$. For block i , we maintain a pair of unit quaternions $(q_L^{(i)}, q_R^{(i)})$ and compute

$$\tilde{v}^{(i)} = q_L^{(i)} v^{(i)} \overline{q_R^{(i)}}, \quad (22)$$

$$\hat{v}^{(i)} = Q(\tilde{v}^{(i)}), \quad (23)$$

$$v_{\text{rec}}^{(i)} = \overline{q_L^{(i)}} \hat{v}^{(i)} q_R^{(i)}. \quad (24)$$

This uses the full six-dimensional rotational freedom of $SO(4)$ and offers the strongest local mixing.

5.3 IsoQuant-Fast

IsoQuant-Fast restricts the transform to a single isoclinic factor:

$$\tilde{v}^{(i)} = q_L^{(i)} v^{(i)}, \quad (25)$$

$$\hat{v}^{(i)} = Q\left(\tilde{v}^{(i)}\right), \quad (26)$$

$$v_{\text{rec}}^{(i)} = \overline{q_L^{(i)}} \hat{v}^{(i)}. \quad (27)$$

Geometrically, this corresponds to a 3-dimensional subgroup of $SO(4)$ isomorphic to $SO(3)$. It sacrifices expressivity for lower parameter count, lower arithmetic cost, and simpler kernels.

5.4 A Lightweight 2D Special Case

Although IsoQuant is fundamentally motivated by blockwise $SO(4)$ structure, the same implementation philosophy admits a degenerate planar special case on coordinate pairs. We partition the normalized input into 2D blocks

$$u^{(j)} \in \mathbb{R}^2, \quad (28)$$

and apply a standard planar rotation

$$\tilde{u}^{(j)} = R(\theta^{(j)}) u^{(j)}, \quad (29)$$

$$\hat{u}^{(j)} = Q\left(\tilde{u}^{(j)}\right), \quad (30)$$

$$u_{\text{rec}}^{(j)} = R(-\theta^{(j)}) \hat{u}^{(j)}, \quad (31)$$

where

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}. \quad (32)$$

We do not present this 2D case as a separate method family; rather, we treat it as an extremely lightweight IsoQuant operating point. Its local mixing capacity is weaker than the 4D formulations, so IsoQuant-Full remains the primary and most expressive variant. Nevertheless, the 2D special case is useful experimentally because it clarifies how much accuracy is lost, if any, when one pushes blockwise rotation cost even lower.

5.5 Parameterization and Learning

To avoid constrained optimization on the manifold directly, we parameterize each unit quaternion by an unconstrained vector and normalize it on the fly:

$$q_L^{(i)} = \frac{u_L^{(i)}}{\|u_L^{(i)}\|_2}, \quad q_R^{(i)} = \frac{u_R^{(i)}}{\|u_R^{(i)}\|_2}, \quad (33)$$

where $u_L^{(i)}, u_R^{(i)} \in \mathbb{R}^4$ are free parameters. This keeps optimization in Euclidean space while enforcing the unit-quaternion constraint implicitly in the computation graph. A practical lightweight variant is to sample the initial u vectors from a Gaussian distribution and keep them fixed, yielding random block rotations analogous to the randomized transform used by TurboQuant.

5.6 Quantization Pipeline

Algorithm 1 summarizes the stage-1 pipeline.

Algorithm 1 IsoQuant Stage-1 Quantization

Require: Input vector $x \in \mathbb{R}^d$, scalar quantizer Q , mode $\in \{\text{FULL}, \text{FAST}, \text{2D}\}$

- 1: Compute norm $\rho \leftarrow \|x\|_2$ and normalized vector $\bar{x} \leftarrow x / \max(\rho, \varepsilon)$
 - 2: Partition $\bar{x}$ into zero-padded local blocks according to the mode (4D for FULL/FAST, 2D for 2D)
 - 3: **for** $i = 1$ to g **do**
 - 4: **if** mode = FULL **then**
 - 5: $\tilde{v}^{(i)} \leftarrow q_L^{(i)} v^{(i)} q_R^{(i)}$
 - 6: $\hat{v}^{(i)} \leftarrow Q(\tilde{v}^{(i)})$
 - 7: $v_{\text{rec}}^{(i)} \leftarrow q_L^{(i)} \hat{v}^{(i)} q_R^{(i)}$
 - 8: **else if** mode = FAST **then**
 - 9: $\tilde{v}^{(i)} \leftarrow q_L^{(i)} v^{(i)}$
 - 10: $\hat{v}^{(i)} \leftarrow Q(\tilde{v}^{(i)})$
 - 11: $v_{\text{rec}}^{(i)} \leftarrow q_L^{(i)} \hat{v}^{(i)}$
 - 12: **else**
 - 13: $\tilde{u}^{(i)} \leftarrow R(\theta^{(i)}) u^{(i)}$
 - 14: $\hat{u}^{(i)} \leftarrow Q(\tilde{u}^{(i)})$
 - 15: $u_{\text{rec}}^{(i)} \leftarrow R(-\theta^{(i)}) \hat{u}^{(i)}$
 - 16: **end if**
 - 17: **end for**
 - 18: Concatenate reconstructed blocks and drop padded coordinates
 - 19: **return** $\hat{x} \leftarrow \rho$ times the concatenated reconstructed blocks
-

6 Complexity Analysis

A quaternion multiplication uses 16 scalar multiplications and 12 scalar additions. To match common systems reporting conventions, we count this as approximately 16 fused multiply-add operations. For $d = 128$, the forward rotation cost becomes straightforward to compare.

Table 1: Forward rotation complexity at $d = 128$.

Method	Block Structure	Params	FMAs
TurboQuant [1]	dense 128×128	16,384	16,384
RotorQuant [2]	$43 \times 3\text{D}$ blocks	172	$\approx 2,408$
IsoQuant-2D	$64 \times 2\text{D}$ blocks	128	≈ 256
IsoQuant-Full	$32 \times 4\text{D}$ blocks	256	1,024
IsoQuant-Fast	$32 \times 4\text{D}$ blocks	128	512

Table 1 summarizes the forward stage-1 rotation cost at $d = 128$. The comparison highlights three regimes. IsoQuant-Full uses somewhat more parameters than RotorQuant because each block stores two quaternions, but it still remains tiny relative to dense rotations and cuts rotation arithmetic by more than $2\times$. IsoQuant-Fast goes further and is cheaper than RotorQuant in both parameters and compute. The 2D special case is cheaper still, but also has the weakest local mixing and should therefore be viewed as a low-cost operating point rather than the main target of the paper.

More generally, if $g_4 = \lceil d/4 \rceil$ and $g_2 = \lceil d/2 \rceil$, then the parameter count is $8g_4$ for Full, $4g_4$ for Fast, and $2g_2$ for the 2D special case. The forward rotation cost is $32g_4$ FMAs for Full, $16g_4$ FMAs for Fast, and approximately $4g_2$ FMAs for the 2D case. All variants therefore scale linearly in d .

7 Systems Considerations

7.1 Why 4D Blocks Are Hardware Friendly

The choice of block size is not purely algebraic. It directly shapes memory layout, vectorization efficiency, and kernel fusion opportunities.

Alignment. Most transformer head dimensions are powers of two. A 4D partition therefore avoids the pathological tails induced by 3D chunking in almost every common setting. At $d = 128$, IsoQuant uses exactly 32 blocks with no remainder, whereas a 3D design requires 42 full blocks plus a leftover fragment.

Vectorization. Four-wide blocks fit naturally into SIMD-friendly load and store patterns such as `float4`. This reduces boundary checks and helps both CPU SIMD backends and GPU kernels maintain regular control flow.

Kernel fusion. The per-block transform is a closed-form sequence of one or two quaternion products, coordinate-wise scalar quantization, and an inverse transform. This structure is especially suitable for fused kernels in online quantization pipelines because the entire block can often remain in registers from input load through output store.

7.2 Prototype Mapping

Our current prototype follows exactly this design pattern: it packs feature vectors into 4D blocks, applies either double-sided or single-sided quaternion rotation, performs scalar Lloyd-Max quantization, and reconstructs the result with a fused CUDA kernel. In addition, we include a lightweight 2D planar special case that serves as an even cheaper operating point within the same implementation family. We built a benchmark harness that directly compares these fused IsoQuant kernels against the fused RotorQuant CUDA kernel under matched tensor shapes, bit widths, and data types.

8 Compatibility with Residual Correction

IsoQuant is intended as a replacement for the stage-1 decorrelation transform, not as a rejection of later residual correction. In two-stage pipelines such as TurboQuant [1], one can keep the residual inner-product correction unchanged:

$$r = x - \hat{x}_{\text{mse}}. \quad (34)$$

The residual can still be projected with a quantized Johnson-Lindenstrauss transform [3] or a related low-bit correction mechanism. In this sense, IsoQuant is complementary to existing stage-2 estimators: it reduces the cost of the orthogonalization step while remaining compatible with unbiased inner-product correction.

9 Experimental Results

9.1 Evaluation Protocol

This section evaluates the stage-1 quantize-dequantize path of IsoQuant. The purpose is to isolate the contribution of the decorrelation transform itself before introducing stage-2 residual correction or downstream task effects. We therefore focus on two directly attributable quantities: reconstruction MSE on normalized synthetic vectors and fused CUDA kernel latency for the blockwise stage-1 transform.

All experiments in this section use synthetic normalized vectors with batch size 8192. We evaluate all combinations of

$$d \in \{128, 256, 512\}, \quad b \in \{2, 3, 4\}, \quad \text{dtype} \in \{\text{fp16}, \text{fp32}\}, \quad (35)$$

for a total of 18 fused-kernel benchmark settings. We focus on these dimensions because they cover common per-head or grouped-KV widths in practical LLM deployments, while avoiding the large-dimension kernel-specialization regime that would otherwise dominate the discussion. On the IsoQuant side, we evaluate IsoQuant-Full, IsoQuant-Fast, and the lightweight IsoQuant-2D special case. On the baseline side, we compare against RotorQuant [2], including its fused CUDA kernel in `rotor_fused_kernel.cu`. TurboQuant [1] is used as a conceptual dense-rotation reference in the complexity analysis, not as a runtime baseline.

All fused CUDA benchmarks were conducted on a single NVIDIA RTX 4090 GPU. For every configuration, RotorQuant and IsoQuant are benchmarked under the same tensor shape, bit width, and execution dtype. In addition to kernel latency, we report reconstruction MSE, parameter count, and estimated forward arithmetic cost.

9.2 Main Results

Table 2 reports the full fused CUDA sweep. The main takeaway is that the entire IsoQuant family is consistently faster than RotorQuant under an apples-to-apples kernel comparison while preserving essentially identical reconstruction quality. Across all 18 settings, IsoQuant-Full achieves an average speedup of $4.49\times$ over RotorQuant fused CUDA, while IsoQuant-Fast and the lightweight IsoQuant-2D special case achieve nearly identical averages of $4.66\times$ and $4.66\times$, respectively. The strongest settings occur in low-bit and medium-width regimes, where speedups exceed $6\times$ while MSE remains unchanged up to the reported precision.

Table 2: Fused CUDA comparisons against RotorQuant on normalized synthetic vectors with batch size 8192. Latencies are in μs .

dtype	bits	dim	RotorQuant	IsoQuant -Full	IsoQuant -Fast	IsoQuant -2D	Full speedup	Fast speedup	2D speedup
fp16	2	128	32.7	8.5	8.2	8.5	3.86	3.98	3.85
fp16	3	128	36.4	6.2	6.1	6.2	5.92	6.00	5.92
fp16	4	128	44.2	9.6	9.4	9.4	4.62	4.71	4.70
fp16	2	256	32.6	8.8	8.3	8.4	3.72	3.92	3.89
fp16	3	256	36.8	9.7	9.5	9.2	3.80	3.88	3.99
fp16	4	256	46.7	8.1	7.5	7.5	5.76	6.24	6.20
fp16	2	512	36.0	8.4	8.5	8.3	4.27	4.23	4.31
fp16	3	512	39.9	9.9	8.4	9.6	4.05	4.73	4.15
fp16	4	512	50.4	13.9	12.5	13.4	3.63	4.02	3.77
fp32	2	128	33.4	7.3	7.1	7.0	4.60	4.68	4.77
fp32	3	128	35.1	6.9	7.6	7.1	5.07	4.59	4.91
fp32	4	128	44.7	8.2	8.2	8.3	5.49	5.48	5.40
fp32	2	256	33.8	7.4	7.2	7.2	4.59	4.66	4.66
fp32	3	256	37.9	7.2	7.0	6.9	5.29	5.45	5.47
fp32	4	256	47.9	8.1	7.6	7.5	5.92	6.31	6.39
fp32	2	512	37.8	12.1	11.4	10.5	3.11	3.32	3.60
fp32	3	512	44.1	12.5	11.5	11.0	3.54	3.85	4.02
fp32	4	512	52.9	14.8	13.5	13.9	3.56	3.91	3.79

The quality story is equally encouraging. In every tested setting, the MSE of all IsoQuant variants is either indistinguishable from RotorQuant or slightly lower. For example, in the largest tested FP32 configuration ($d = 512, b = 4$), all three IsoQuant variants remain at the same numerical scale as RotorQuant, and in many settings the printed MSE values are exactly identical. We therefore do not observe a tradeoff in which IsoQuant wins speed by degrading reconstruction quality.

9.3 Result Interpretation

The full sweep reveals three robust trends.

1. **IsoQuant-Fast and IsoQuant-2D are the lowest-latency operating points.** Their mean speedups are nearly identical, and each variant attains the best latency in a subset of the tested settings. We therefore view the 2D case as a useful lightweight special case rather than a replacement for the 4D family.
2. **The gain is strong in both FP16 and FP32.** Averaged over all tested FP16 settings, IsoQuant-Full, IsoQuant-Fast, and IsoQuant-2D achieve $4.40\times$, $4.63\times$, and $4.53\times$ speedups, respectively; the corresponding FP32 averages are $4.57\times$, $4.69\times$, and $4.78\times$.
3. **Low-bit and medium-width settings are especially favorable.** Several configurations in the ($d = 128, 256$) range exceed $6\times$ speedup, showing that the compact blockwise formulations amortize per-block overhead particularly well in practical KV-cache regimes.

These trends are consistent with the systems argument developed earlier. Compared with RotorQuant’s 3D Clifford blocks, IsoQuant avoids the expansion to an 8-component multivector representation, keeps the per-block state smaller, and aligns naturally with two-wide or four-wide memory and register organization. The result is not a change in asymptotic complexity—all methods remain linear in d —but a real and repeatable reduction in constant factors.

9.4 Module-Level vs Kernel-Level Measurements

In addition to fused-kernel measurements, we also benchmarked higher-level PyTorch module paths. There, the apparent speedups can reach roughly $4\times$ – $10\times$, because the IsoQuant prototype currently enjoys a more streamlined execution path than the baseline RotorQuant module. However, we regard the fused CUDA comparison as the fairest measure of method-intrinsic systems advantage. The fused results therefore serve as our primary hardware claim, while the larger module-level gains should be interpreted as an implementation-dependent systems outcome.

9.5 Ablation Coverage

Table 3: Ablation axes. Completed settings in this work are marked by checkmarks.

Axis	Values	
Rotation type	Full, Fast, 2D special case	✓
Bit width	2, 3, 4 bits	✓
Block size	3D, 4D, optionally 8D grouped variants	
Quaternion parameters	random fixed ✓, learned normalized	
Residual correction	off ✓, QJL-style correction on	
Deployment backend	PyTorch ✓, fused CUDA	✓

Table 3 summarizes the experimental axes covered by the current draft. The completed experiments already establish a strong stage-1 result across rotation type, bit width, and backend, but they do not yet include residual correction or downstream KV-cache task metrics.

9.6 What Remains for a Full Submission

The current experiments establish a strong stage-1 result, but a full submission should still add downstream KV-cache metrics:

1. integration with a stage-2 residual correction module such as QJL;
2. attention-logit preservation and inner-product error under the complete two-stage pipeline;
3. retrieval and perplexity experiments on real KV tensors extracted from deployed LLMs.

These follow naturally from the present implementation because IsoQuant only replaces the stage-1 transform and remains compatible with the same residual correction machinery used by TurboQuant and RotorQuant.

10 Limitations and Future Work

IsoQuant is not a full solution by itself.

1. **Block locality.** Like other block-diagonal transforms, it does not mix information across blocks, so global correlations remain unaddressed.
2. **Evaluation gap.** This draft now includes fused CUDA benchmarks and reconstruction experiments, but the final conference version still needs end-to-end KV-cache validation on real models and tasks.
3. **Learning dynamics.** Although normalized quaternion parameters are simple to optimize, the relative value of learned versus random rotations remains an empirical question.
4. **Stage-2 interaction.** The best coupling between 4D block decorrelation and residual correction may depend on the downstream attention estimator.

Promising next steps include hierarchical cross-block mixing, jointly learned codebooks and quaternion parameters, and specialized kernels for fused KV-cache compression during autoregressive decoding.

11 Conclusion

IsoQuant replaces awkward 3D block rotations with a hardware-aligned 4D quaternion formulation grounded in the isoclinic decomposition of $SO(4)$. The resulting design retains the spirit of blockwise decorrelation while offering stronger local mixing, cleaner memory alignment, and lower arithmetic cost. Our fused CUDA experiments on practical dimensions $d \in \{128, 256, 512\}$ show that this design advantage is measurable in practice: across 18

matched settings, IsoQuant-Full, IsoQuant-Fast, and the lightweight 2D special case deliver average speedups of $4.49\times$, $4.66\times$, and $4.66\times$ over RotorQuant fused CUDA while preserving essentially identical reconstruction MSE. We therefore view the 2D case as a useful low-cost operating point, while the full 4D construction remains the primary and most expressive form of the method. These results make a strong case that blockwise isoclinic rotations are a compelling stage-1 replacement for online vector quantization, and they motivate the next step of integrating the method with residual correction and full KV-cache evaluation. An additional advantage of quaternion-pair parameterization is that it admits smooth interpolation on the underlying rotation manifold, which may be useful for shared or adaptive block rotations in future work.

References

- [1] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni. TurboQuant: Online vector quantization with near-optimal distortion rate. In *International Conference on Learning Representations*, 2026.
- [2] J. D. Pope. RotorQuant: Clifford algebra vector quantization for LLM KV cache compression. 2026. <https://www.scrya.com/rotorquant/>. Code: <https://github.com/scrya-com/rotorquant>.
- [3] A. Zandieh, M. Daliri, and I. Han. QJL: 1-bit quantized JL transform for KV cache quantization with zero overhead. *arXiv preprint arXiv:2406.03482*, 2024.
- [4] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. *arXiv preprint arXiv:2402.02750*, 2024.
- [5] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.
- [6] J. B. Kuipers. *Quaternions and Rotation Sequences*. Princeton University Press, 1999.
- [7] Z. Ji. RiemannFormer: A framework for attention in curved spaces. *arXiv preprint arXiv:2506.07405*, 2025.